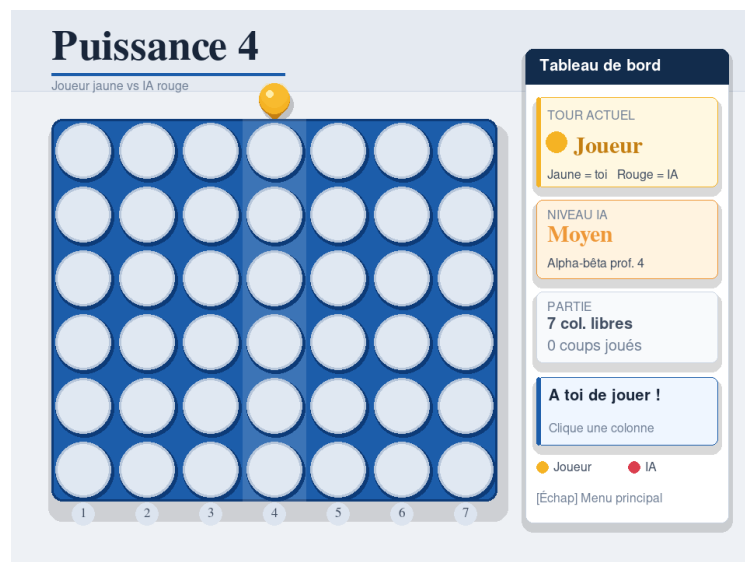


Université Moulay Ismaïl
Faculté des Sciences et Techniques Errachidia
Cycle d'Ingénieur d'État
Filière : **Ingénierie Logicielle et Intelligence Artificielle**

Compte rendu de mini-projet
Intelligence Artificielle

Puissance 4

Jeu de stratégie avec agent intelligent
Algorithme Min-Max avec élagage Alpha-Bêta



Réalisé par :

Laghrib Salim
Zouhri Salma

Encadré par :

Pr. Khaddouj Taifi

Année

universitaire : 2025–2026

Résumé

Ce compte rendu présente la conception et la réalisation d'un jeu de stratégie **Puissance 4** intégrant un agent intelligent. Le projet répond au cahier des charges du mini-projet d'intelligence artificielle : présenter la méthode utilisée, étudier sa complexité, implémenter l'algorithme en langage Python et fournir une interface graphique valorisant l'application.

Le système développé permet à un joueur humain d'affronter une intelligence artificielle. L'agent choisit ses coups à partir de l'algorithme **Min-Max**, puis d'une version optimisée par **élagage alpha-bêta**. Comme l'arbre complet du Puissance 4 est trop grand pour être exploré entièrement, l'algorithme utilise une profondeur limitée et une fonction heuristique d'évaluation du plateau. Cette heuristique favorise les positions centrales, les alignements prometteurs et les coups qui bloquent les menaces adverses.

Le projet est structuré en modules : gestion du plateau, logique de l'IA, interface graphique et lancement de l'application. L'interface est réalisée avec **Pygame**, tandis que le plateau est représenté par une matrice **NumPy**. Les tests effectués montrent que l'IA sait gagner lorsqu'un coup immédiat est disponible, bloquer une menace directe et adapter son niveau de difficulté selon la profondeur de recherche.

Mots-clés : Intelligence artificielle, jeu de stratégie, Puissance 4, Min-Max, élagage alpha-bêta, heuristique, Python, Pygame.

Table des matières

1	Introduction et présentation de la méthode	1
1.1	Objectifs du projet	1
1.2	Choix du mini-projet	1
1.3	Présentation générale du jeu	2
1.3.1	Règles du Puissance 4	2
1.3.2	Modélisation du problème	2
1.3.3	Contraintes du projet	3
1.4	Méthode d'intelligence artificielle	3
1.4.1	Principe de l'algorithme Min-Max	3
1.4.2	Pseudo-code de Min-Max	4
1.4.3	Limitation par profondeur	4
1.5	Élagage alpha-bêta	4
1.5.1	Motivation	4
1.5.2	Définition de alpha et bêta	4
1.5.3	Pseudo-code de alpha-bêta	5
1.5.4	Intérêt pratique dans notre jeu	5
1.6	Fonction heuristique d'évaluation	5
1.6.1	Rôle de l'heuristique	5
1.6.2	Fenêtres de quatre cases	5
1.6.3	Pourquoi favoriser le centre ?	6
2	Complexité de la méthode	6
2.1	Complexité de Min-Max	6
2.2	Complexité avec alpha-bêta	7
2.3	Comparaison théorique	7
3	Implémentation de l'algorithme en Python pour résoudre un jeu de stratégie	7
3.1	Technologies utilisées	7
3.2	Organisation des fichiers	8
3.3	Représentation du plateau	8
3.4	Détection de victoire	8
3.4.1	Principe	8
3.4.2	Exemple de vérification horizontale	8
3.4.3	Gestion du match nul	9
3.5	Interface graphique	9
3.5.1	Choix de Pygame	9
3.5.2	Niveaux de difficulté	10
3.6	Stratégie de l'IA	10
3.6.1	IA simple	10
3.6.2	IA avec alpha-bêta	10
3.6.3	Gestion des coups valides	11
3.7	Tests et résultats	11
3.7.1	Tests fonctionnels	11
3.7.2	Exemple de blocage	11
3.7.3	Exemple de victoire	12
3.8	Analyse des résultats	12
3.8.1	Comportement observé	12
3.8.2	Impact de la profondeur	13

3.8.3	Forces du projet	13
3.8.4	Limites	13
3.9	Difficultés rencontrées	13
3.9.1	Gestion de l'explosion combinatoire	13
3.9.2	Conception de l'heuristique	13
3.9.3	Synchronisation avec l'interface	14
3.10	Perspectives d'amélioration	14
3.10.1	Mémoïsation des états	14
3.10.2	Amélioration de l'ordre des coups	14
3.10.3	Modes de jeu supplémentaires	14
3.10.4	Comparaison avec d'autres méthodes	14
4	Conclusion	14

1 Introduction et présentation de la méthode

L'intelligence artificielle occupe une place importante dans les jeux de stratégie, car ces derniers offrent un cadre clair pour étudier la prise de décision, l'anticipation et l'optimisation. Dans ce mini-projet, nous avons choisi de réaliser un jeu de **Puissance 4** dans lequel un joueur humain affronte un agent intelligent.

Le Puissance 4 est un jeu à deux joueurs qui se déroule sur une grille de six lignes et sept colonnes. À chaque tour, un joueur choisit une colonne ; le jeton tombe alors automatiquement dans la case libre la plus basse de cette colonne. L'objectif est d'aligner quatre jetons identiques horizontalement, verticalement ou diagonalement avant l'adversaire.

Ce jeu est particulièrement adapté à l'étude de l'algorithme Min-Max. Chaque décision peut être représentée comme un arbre de possibilités : l'IA joue un coup, le joueur répond, puis l'IA rejoue, et ainsi de suite. L'agent intelligent doit donc anticiper les conséquences de ses actions tout en supposant que l'adversaire cherchera lui aussi à jouer de manière rationnelle.

Le travail réalisé respecte le plan demandé dans la consigne du mini-projet : une introduction avec présentation de la méthode, une étude de complexité, une implémentation de l'algorithme en Python pour résoudre un problème de jeu de stratégie, puis une conclusion. Le projet correspond à la catégorie **jeu de stratégie** et utilise l'algorithme **Min-Max** avec élagage alpha-bêta.

1.1 Objectifs du projet

Les objectifs principaux de ce mini-projet sont les suivants :

- modéliser correctement les règles du Puissance 4 ;
- représenter le plateau sous une forme exploitable par un programme ;
- implémenter un agent intelligent capable de choisir un coup pertinent ;
- étudier la complexité de la méthode utilisée ;
- proposer une interface graphique claire pour jouer contre l'IA ;
- tester le comportement de l'agent dans des situations de victoire et de blocage.

1.2 Choix du mini-projet

Dans la liste proposée pour le module, notre projet correspond à la catégorie **jeu de stratégie**. Nous avons retenu le Puissance 4 parce que ses règles sont simples à comprendre, mais sa résolution nécessite une vraie stratégie. Le jeu demande d'évaluer plusieurs coups à l'avance, d'identifier les menaces adverses et de contrôler les zones importantes du plateau.

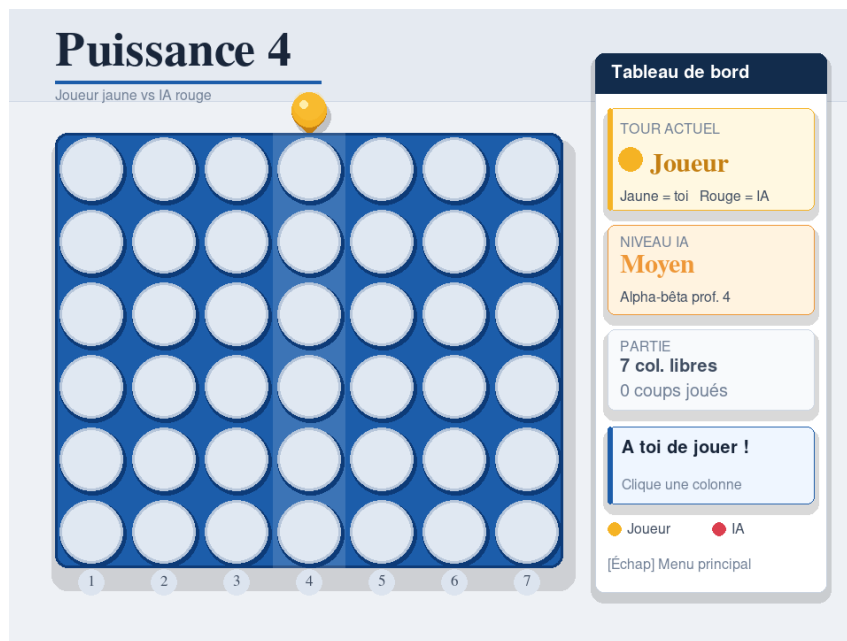


FIGURE 1 – Interface d’accueil du jeu développé

1.3 Présentation générale du jeu

1.3.1 Règles du Puissance 4

Le Puissance 4 se joue sur une grille de 6×7 , soit 42 cases. Deux joueurs placent alternativement leurs jetons dans une colonne. En raison de la gravité, un jeton ne peut pas être placé n’importe où : il occupe automatiquement la position libre la plus basse de la colonne choisie.

Une partie se termine dans l’un des cas suivants :

- un joueur aligne quatre jetons de sa couleur ;
- le plateau est rempli sans alignement gagnant, ce qui produit un match nul.

Les alignements gagnants peuvent apparaître dans quatre directions : horizontale, verticale, diagonale montante ou diagonale descendante. La détection de ces alignements est donc une partie essentielle de la logique du jeu.

1.3.2 Modélisation du problème

Un état du jeu est défini par le contenu du plateau et par le joueur dont c’est le tour. Dans notre implémentation, chaque case du plateau prend l’une des trois valeurs suivantes :

Valeur	Signification
0	Case vide
1	Jeton du joueur humain
2	Jeton de l’intelligence artificielle

TABLE 1 – Codage des cases du plateau

Le choix d’une matrice est naturel, car il correspond directement à la structure de la grille. Il facilite également le parcours des lignes, colonnes et diagonales pour détecter les victoires et calculer le score heuristique.

1.3.3 Contraintes du projet

Selon la consigne fournie pour le mini-projet, le compte rendu doit inclure une introduction avec présentation de la méthode, une analyse de la complexité, une implémentation de l'algorithme dans un langage de programmation, ainsi qu'une conclusion. La réalisation d'une interface graphique est recommandée afin de valoriser l'application.

Notre projet répond à ces attentes de la manière suivante :

- une implémentation Python complète ;
- une interface graphique interactive avec **Pygame** ;
- une IA fondée sur **Min-Max** et l'élagage **alpha-bêta** ;
- une séparation claire entre logique du jeu, IA et affichage ;
- des captures d'écran montrant les principaux états de l'application.

1.4 Méthode d'intelligence artificielle

1.4.1 Principe de l'algorithme Min-Max

L'algorithme Min-Max est utilisé dans les jeux à deux joueurs, à somme nulle, dans lesquels l'amélioration de la situation d'un joueur correspond à la dégradation de la situation de l'autre. Dans notre projet, l'IA est le joueur maximisant, tandis que l'humain est considéré comme le joueur minimisant.

L'idée générale est la suivante : pour chaque coup possible de l'IA, le programme simule les réponses possibles du joueur, puis les coups suivants de l'IA, jusqu'à une profondeur limitée. Les feuilles de cet arbre sont évaluées par une fonction de score. L'IA choisit ensuite le coup qui maximise son résultat en supposant que l'adversaire jouera de façon optimale.

Élément	Rôle dans l'algorithme
Nœud	État possible du plateau
Branche	Coup possible dans une colonne valide
Niveau max	Tour de l'IA, qui cherche le meilleur score
Niveau min	Tour du joueur humain, qui cherche à réduire le score de l'IA
Feuille	État terminal ou état atteint à la profondeur maximale

TABLE 2 – Interprétation de l'arbre de jeu

1.4.2 Pseudo-code de Min-Max

Le fonctionnement de Min-Max peut être résumé par le pseudo-code suivant :

Listing 1 – Pseudo-code simplifié de Min-Max

```
fonction minimax(plateau, profondeur, ia_joue):
    si plateau terminal ou profondeur = 0:
        retourner evaluation(plateau)

    si ia_joue:
        meilleur_score = -infini
        pour chaque colonne valide:
            simuler le coup de l'IA
            score = minimax(nouveau_plateau, profondeur - 1, faux)
            meilleur_score = max(meilleur_score, score)
        retourner meilleur_score

    sinon:
        meilleur_score = +infini
        pour chaque colonne valide:
            simuler le coup du joueur
            score = minimax(nouveau_plateau, profondeur - 1, vrai)
            meilleur_score = min(meilleur_score, score)
        retourner meilleur_score
```

1.4.3 Limitation par profondeur

Même si le Puissance 4 paraît simple, l'arbre de jeu devient rapidement très grand. Il n'est donc pas réaliste d'explorer toutes les parties jusqu'à la fin. Nous utilisons une profondeur limitée : lorsque cette profondeur est atteinte, l'algorithme ne connaît pas encore forcément le gagnant, mais il estime la qualité du plateau grâce à une heuristique.

Cette approche constitue un compromis entre qualité de décision et temps d'exécution. Une profondeur faible rend l'IA rapide, mais moins stratégique. Une profondeur élevée rend l'IA plus forte, mais augmente fortement le temps de calcul.

1.5 Élagage alpha-bêta

1.5.1 Motivation

Le Min-Max simple explore toutes les branches de l'arbre jusqu'à la profondeur fixée. Or certaines branches ne peuvent pas modifier la décision finale. L'élagage alpha-bêta permet d'éviter l'exploration de ces branches inutiles.

Cette optimisation ne change pas le résultat théorique de Min-Max : elle permet d'obtenir le même choix, mais en visitant moins d'états lorsque l'ordre d'exploration est favorable.

1.5.2 Définition de alpha et bêta

Deux bornes sont utilisées pendant la recherche :

- α représente le meilleur score déjà garanti pour le joueur maximisant, c'est-à-dire l'IA ;
- β représente le meilleur score déjà garanti pour le joueur minimisant, c'est-à-dire le joueur humain.

Lorsque la condition $\alpha \geq \beta$ est vérifiée, l'algorithme peut arrêter l'exploration de la branche courante. En effet, le joueur rationnel ne choisira pas une branche qui mène à un résultat moins favorable que ce qu'il peut déjà garantir ailleurs.

1.5.3 Pseudo-code de alpha-bêta

Listing 2 – Pseudo-code de Min-Max avec élagage alpha-bêta

```
fonction alphabeta(plateau, profondeur, alpha, beta, ia_joue):
    si plateau terminal ou profondeur = 0:
        retourner evaluation(plateau)

    si ia_joue:
        valeur = -infini
        pour chaque colonne valide:
            simuler le coup de l'IA
            valeur = max(valeur, alphabeta(nouveau_plateau,
                                           profondeur - 1,
                                           alpha, beta, faux))

            alpha = max(alpha, valeur)
            si alpha >= beta:
                arreter la boucle
        retourner valeur

    sinon:
        valeur = +infini
        pour chaque colonne valide:
            simuler le coup du joueur
            valeur = min(valeur, alphabeta(nouveau_plateau,
                                           profondeur - 1,
                                           alpha, beta, vrai))

            beta = min(beta, valeur)
            si alpha >= beta:
                arreter la boucle
        retourner valeur
```

1.5.4 Intérêt pratique dans notre jeu

Dans l'application, alpha-bêta permet d'utiliser une profondeur de recherche plus élevée pour les niveaux moyen et expert. Cela améliore la capacité de l'IA à anticiper les coups adverses tout en gardant une interaction fluide avec l'utilisateur.

1.6 Fonction heuristique d'évaluation

1.6.1 Rôle de l'heuristique

Lorsque la profondeur maximale est atteinte sans victoire immédiate, l'IA doit tout de même comparer les états du plateau. Pour cela, nous utilisons une fonction heuristique qui attribue un score numérique à chaque position.

Cette fonction ne garantit pas de connaître le résultat final de la partie, mais elle donne une estimation utile. Une position est jugée favorable si elle augmente les possibilités de victoire de l'IA ou réduit celles du joueur.

1.6.2 Fenêtres de quatre cases

Le principe retenu consiste à parcourir toutes les fenêtres de quatre cases dans les quatre directions possibles. Chaque fenêtre est évaluée en fonction du nombre de jetons de l'IA, du nombre de jetons du joueur et du nombre de cases vides.

Situation détectée	Score
4 jetons alignés pour l'IA	+100000
3 jetons IA et 1 case vide	+50
2 jetons IA et 2 cases vides	+2
4 jetons alignés pour le joueur	-100000
3 jetons joueur et 1 case vide	-80
Jetons de l'IA dans la colonne centrale	+3

TABLE 3 – Principaux critères de la fonction d'évaluation

1.6.3 Pourquoi favoriser le centre ?

Dans le Puissance 4, la colonne centrale est généralement stratégique. Un jeton placé au centre participe à davantage de combinaisons potentielles qu'un jeton placé sur les bords. C'est pourquoi notre heuristique ajoute un bonus lorsque l'IA occupe la colonne centrale.

Listing 3 – Extrait de la fonction d'évaluation

```
def evaluer_fenetre(fenetre, piece):
    adversaire = JOUEUR if piece == IA else IA
    score = 0

    nb_piece = fenetre.count(piece)
    nb_vide = fenetre.count(VIDE)
    nb_adv = fenetre.count(adversaire)

    if nb_piece == 4:
        score += 100000
    elif nb_piece == 3 and nb_vide == 1:
        score += 50
    elif nb_piece == 2 and nb_vide == 2:
        score += 2

    if nb_adv == 4:
        score -= 100000
    elif nb_adv == 3 and nb_vide == 1:
        score -= 80

    return score
```

2 Complexité de la méthode

2.1 Complexité de Min-Max

Dans le Puissance 4, le facteur de branchement maximal est $b = 7$, car un joueur peut choisir au maximum sept colonnes. Si la profondeur de recherche est d , le nombre maximal d'états explorés par Min-Max simple est :

$$1 + b + b^2 + \dots + b^d$$

En notation asymptotique, cela donne :

$$O(b^d) = O(7^d)$$

Par exemple, pour une profondeur $d = 5$, le nombre de feuilles possibles est :

$$7^5 = 16807$$

Ce nombre augmente exponentiellement avec la profondeur, ce qui explique pourquoi une recherche complète serait trop coûteuse.

2.2 Complexité avec alpha-bêta

L'élagage alpha-bêta améliore fortement les performances lorsque les bons coups sont examinés tôt. Dans le meilleur cas, la complexité peut être rapprochée de :

$$O(b^{d/2})$$

Pour le Puissance 4, cela signifie qu'une recherche de profondeur 6 peut se rapprocher, dans un cas favorable, du coût d'une recherche simple de profondeur 3 :

$$7^3 = 343$$

Cette amélioration est très importante pour une application interactive, car le joueur attend une réponse rapide après chaque coup.

2.3 Comparaison théorique

Profondeur	Min-Max simple 7^d	Alpha-bêta idéal $7^{d/2}$	Observation
2	49	7	faible coût
4	2401	49	gain important
6	117649	343	écart très élevé
8	5764801	2401	Min-Max simple impraticable

TABLE 4 – Comparaison indicative du nombre d'états explorés

Dans la pratique, le nombre exact de nœuds explorés dépend de l'état du plateau, du nombre de colonnes encore jouables et de l'ordre dans lequel les coups sont examinés. Cependant, l'analyse montre clairement que l'élagage alpha-bêta est nécessaire pour rendre l'IA plus performante.

3 Implémentation de l'algorithme en Python pour résoudre un jeu de stratégie

3.1 Technologies utilisées

Le projet est développé en langage `Python`. Les principales bibliothèques utilisées sont :

- `NumPy`, pour représenter le plateau sous forme de matrice ;
- `Pygame`, pour créer l'interface graphique et gérer les événements ;
- `random` et `time`, pour certains choix simples et les mesures de performance ;
- `math`, pour le dessin de certains éléments graphiques.

3.2 Organisation des fichiers

Le code est organisé de manière modulaire afin de séparer les responsabilités.

Fichier	Rôle
<code>main.py</code>	Point d'entrée de l'application et lancement du jeu
<code>game.py</code>	Gestion de l'interface graphique, des événements et de la boucle principale
<code>board.py</code>	Fonctions liées au plateau : création, placement, victoire, score
<code>ai.py</code>	Implémentation de Min-Max, alpha-bêta et choix des coups de l'IA

TABLE 5 – Architecture générale du projet

3.3 Représentation du plateau

La fonction `creer_plateau` initialise une matrice de zéros. Chaque ligne et chaque colonne correspondent à une position réelle dans la grille du jeu.

Listing 4 – Création du plateau

```
def creer_plateau():
    plateau = np.zeros((ROWS, COLS), dtype=int)
    return plateau
```

Le placement d'un jeton respecte la gravité : le programme parcourt la colonne depuis le bas vers le haut afin de trouver la première case libre.

Listing 5 – Recherche de la ligne libre dans une colonne

```
def obtenir_ligne_libre(plateau, col):
    for ligne in range(ROWS - 1, -1, -1):
        if plateau[ligne][col] == VIDE:
            return ligne
    return None
```

3.4 Détection de victoire

3.4.1 Principe

La détection de victoire consiste à vérifier si un joueur possède quatre jetons consécutifs. Comme les alignements peuvent être orientés différemment, la fonction `verifier_victoire` parcourt le plateau selon quatre directions.

- horizontalement, de gauche à droite ;
- verticalement, de haut en bas ;
- diagonalement dans le sens montant ;
- diagonalement dans le sens descendant.

3.4.2 Exemple de vérification horizontale

Listing 6 – Vérification horizontale d'un alignement

```
for ligne in range(ROWS):
    for col in range(COLS - 3):
```

```

if (plateau[ligne][col] == piece and
    plateau[ligne][col + 1] == piece and
    plateau[ligne][col + 2] == piece and
    plateau[ligne][col + 3] == piece):
    return True

```

La limite `COLS - 3` permet d'éviter de dépasser les bornes de la matrice. Le même principe est appliqué aux autres directions, avec des indices adaptés.

3.4.3 Gestion du match nul

Un match nul se produit lorsque toutes les colonnes sont pleines et qu'aucun joueur n'a gagné. Cette condition est testée en vérifiant qu'il n'existe plus de colonne valide.

Listing 7 – Test de plateau plein

```

def plateau_plein(plateau):
    return len(colonnes_valides(plateau)) == 0

```

Cette fonction est utilisée dans l'algorithme Min-Max comme condition terminale. Si le plateau est plein, le score retourné est nul, car aucun joueur n'a obtenu d'avantage décisif.

3.5 Interface graphique

3.5.1 Choix de Pygame

L'interface graphique a été réalisée avec `Pygame`. Ce choix permet de créer une fenêtre interactive, d'afficher le plateau, de dessiner les jetons et de gérer les clics de souris. L'utilisateur peut ainsi jouer naturellement en sélectionnant une colonne.

L'interface contient plusieurs éléments :

- une zone principale contenant la grille du Puissance 4 ;
- des jetons de couleurs différentes pour le joueur et l'IA ;
- un panneau latéral indiquant l'état de la partie ;
- des boutons pour choisir le niveau et relancer une partie.

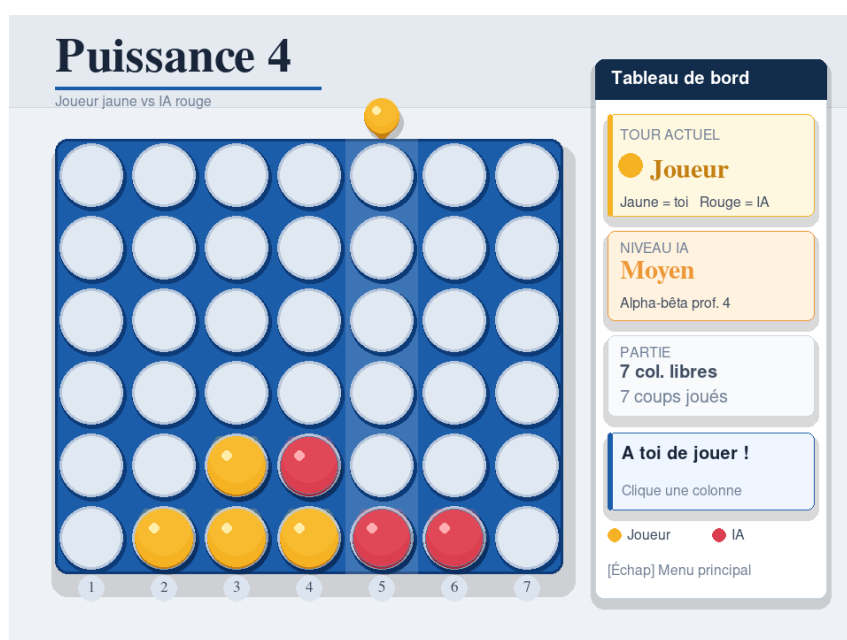


FIGURE 2 – Interface pendant une partie

3.5.2 Niveaux de difficulté

Le jeu propose trois niveaux :

Niveau	Méthode	Description
Simple	Stratégie rapide	L'IA gagne si possible, bloque les menaces et privilégie le centre
Moyen	Alpha-bêta profondeur 4	Compromis entre rapidité et anticipation
Expert	Alpha-bêta profondeur 6	Recherche plus profonde et adversaire plus difficile

TABLE 6 – Niveaux de difficulté proposés



FIGURE 3 – Choix du niveau de difficulté

3.6 Stratégie de l'IA

3.6.1 IA simple

Le niveau simple repose sur une stratégie directe. L'IA commence par vérifier si elle peut gagner immédiatement. Si ce n'est pas possible, elle vérifie si le joueur dispose d'un coup gagnant au tour suivant et bloque ce coup. Ensuite, elle privilégie la colonne centrale ou une colonne proche du centre.

Cette stratégie est rapide et donne un comportement raisonnable pour un niveau débutant, mais elle ne permet pas d'anticiper plusieurs coups à l'avance.

3.6.2 IA avec alpha-bêta

Les niveaux moyen et expert utilisent `minimax_alphabeta`. Pour chaque colonne valide, l'IA simule le coup, explore les réponses possibles du joueur et évalue les positions obtenues. Le meilleur coup est celui qui maximise le score final estimé.

Listing 8 – Appel de l'IA alpha-bêta

```
def coup_ia_alphabeta(plateau, profondeur=5):  
    col, _ = minimax_alphabeta(plateau, profondeur,  
                               -999999, 999999, True)  
    return col
```

3.6.3 Gestion des coups valides

L'IA ne peut jouer que dans une colonne non pleine. La fonction `colonnes_valides` retourne donc la liste des colonnes disponibles. Cela évite de simuler des coups impossibles.

Listing 9 – Liste des colonnes jouables

```
def colonnes_valides(plateau):  
    return [c for c in range(COLS) if colonne_valide(plateau, c)]
```

Cette liste est utilisée à chaque niveau de l'arbre de recherche. Ainsi, plus la partie avance, plus le facteur de branchement réel peut diminuer, car certaines colonnes deviennent pleines.

3.7 Tests et résultats

3.7.1 Tests fonctionnels

Plusieurs tests ont été réalisés pour vérifier le bon fonctionnement du projet :

- création correcte d'un plateau vide ;
- placement des jetons dans la ligne libre la plus basse ;
- détection des victoires horizontales, verticales et diagonales ;
- choix d'un coup gagnant par l'IA lorsqu'il existe ;
- blocage d'une menace immédiate du joueur ;
- fonctionnement des niveaux de difficulté ;
- affichage correct de l'interface graphique.

3.7.2 Exemple de blocage

Dans un scénario de test, le joueur possède trois jetons alignés et peut gagner au coup suivant. L'IA doit alors placer son jeton dans la colonne permettant de bloquer l'alignement. Ce comportement confirme que la fonction d'évaluation et la recherche Min-Max prennent bien en compte les menaces adverses.

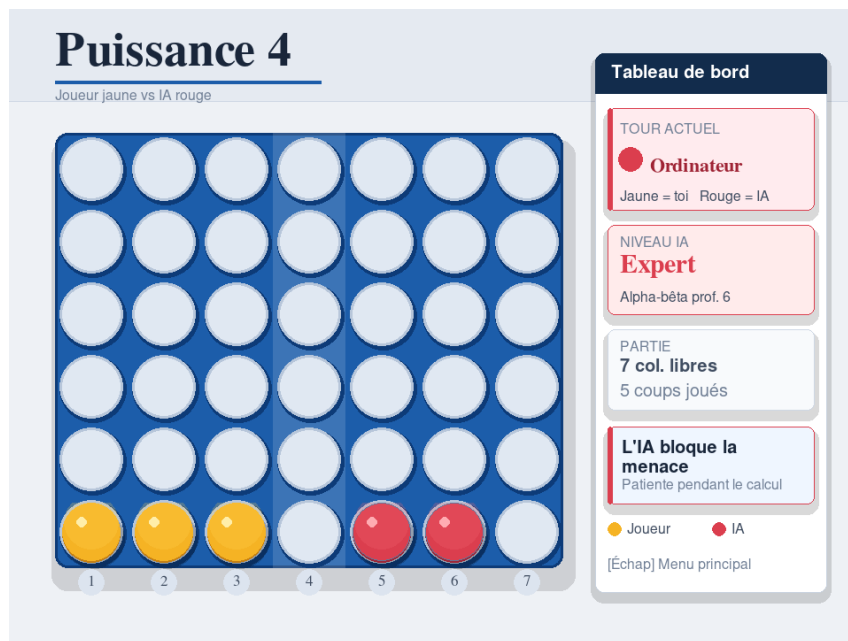


FIGURE 4 – Exemple où l'IA bloque une menace du joueur

3.7.3 Exemple de victoire

Lorsqu'un coup permet à l'IA ou au joueur d'aligner quatre jetons, le programme détecte la victoire et met fin à la partie. L'interface affiche alors le résultat de façon claire.

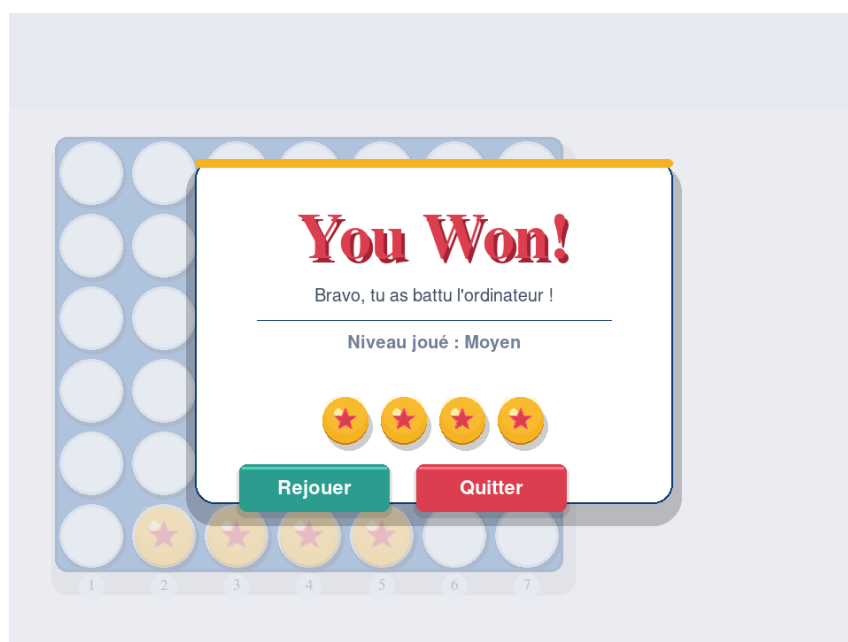


FIGURE 5 – Affichage d'une victoire

3.8 Analyse des résultats

3.8.1 Comportement observé

Les résultats obtenus montrent que le jeu est fonctionnel et que l'IA adopte un comportement cohérent. Au niveau simple, elle est rapide et capable de bloquer les menaces évidentes. Aux

niveaux moyen et expert, elle devient plus stratégique grâce à l'exploration de plusieurs coups futurs.

L'utilisation de la colonne centrale apparaît souvent dans les décisions de l'IA, car l'heuristique lui attribue un bonus. Ce choix améliore généralement les possibilités d'alignement, surtout en début de partie.

3.8.2 Impact de la profondeur

La profondeur de recherche influence fortement la qualité du jeu :

- profondeur faible : réponse rapide, mais anticipation limitée ;
- profondeur moyenne : bon équilibre entre performance et stratégie ;
- profondeur élevée : meilleure anticipation, mais coût de calcul plus important.

Dans notre application, la profondeur 4 est utilisée pour le niveau moyen et la profondeur 6 pour le niveau expert. Ce choix permet de garder un temps de réponse acceptable tout en améliorant la difficulté.

3.8.3 Forces du projet

Les principaux points forts du projet sont :

- une séparation claire entre la logique du jeu et l'interface ;
- une IA basée sur une méthode classique d'intelligence artificielle ;
- une optimisation alpha-bêta permettant de réduire les calculs ;
- une interface graphique complète et agréable à utiliser ;
- des niveaux de difficulté adaptés à différents joueurs.

3.8.4 Limites

Malgré ces résultats, certaines limites existent. L'heuristique reste simple et ne détecte pas toutes les configurations stratégiques avancées. De plus, l'ordre d'exploration des coups pourrait être amélioré pour augmenter l'efficacité de l'élagage alpha-bêta. Enfin, l'IA ne mémorise pas les états déjà analysés, ce qui peut entraîner des calculs répétitifs.

3.9 Difficultés rencontrées

3.9.1 Gestion de l'explosion combinatoire

La principale difficulté rencontrée concerne la taille de l'arbre de recherche. Même avec seulement sept colonnes, le nombre de possibilités augmente très rapidement. Une profondeur trop élevée peut ralentir le jeu, surtout si l'algorithme explore beaucoup de branches.

L'élagage alpha-bêta a permis de réduire cette difficulté, mais son efficacité dépend de l'ordre d'exploration. Si les bons coups sont étudiés en premier, davantage de branches sont coupées.

3.9.2 Conception de l'heuristique

Une autre difficulté importante concerne la fonction de score. Une bonne heuristique doit encourager les coups offensifs, mais aussi empêcher les menaces adverses. Dans notre cas, nous avons attribué un score négatif important aux situations où le joueur possède trois jetons et une case vide. Cela force l'IA à bloquer les menaces immédiates.

Le choix des valeurs numériques est également important. Si un bonus est trop faible, il n'influence presque pas la décision. S'il est trop fort, il peut conduire l'IA à ignorer d'autres situations plus urgentes.

3.9.3 Synchronisation avec l'interface

L'interface graphique doit rester fluide pendant que l'IA réfléchit. Il faut donc trouver un compromis entre profondeur de recherche et temps d'attente. Les niveaux proposés répondent à cette contrainte : le niveau simple est instantané, le niveau moyen reste rapide et le niveau expert donne une IA plus forte.

3.10 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées pour rendre le projet plus complet.

3.10.1 Mémoïsation des états

L'IA peut rencontrer plusieurs fois le même état du plateau par des ordres de coups différents. Une table de transposition permettrait de mémoriser les scores déjà calculés et d'éviter certains recalculs.

3.10.2 Amélioration de l'ordre des coups

L'élagage alpha-bêta est plus efficace lorsque les bons coups sont examinés en premier. Pour le Puissance 4, on peut explorer les colonnes dans l'ordre suivant :

3, 2, 4, 1, 5, 0, 6

Cet ordre favorise le centre du plateau et peut augmenter le nombre de branches coupées.

3.10.3 Modes de jeu supplémentaires

Le projet pourrait être enrichi par :

- un mode joueur contre joueur ;
- un historique des coups ;
- un système de score sur plusieurs parties ;
- une animation plus détaillée de la chute des jetons ;
- un réglage manuel de la profondeur de recherche.

3.10.4 Comparaison avec d'autres méthodes

Il serait également intéressant de comparer Min-Max avec d'autres approches d'intelligence artificielle, comme Monte Carlo Tree Search ou l'apprentissage par renforcement. Ces méthodes peuvent être utiles lorsque l'espace de recherche est très grand ou lorsque l'on souhaite apprendre une stratégie à partir de nombreuses parties simulées.

4 Conclusion

Ce mini-projet nous a permis d'appliquer concrètement des notions importantes d'intelligence artificielle à un jeu de stratégie. Le Puissance 4 constitue un bon support pour comprendre la recherche adversariale, car chaque décision doit tenir compte des coups possibles de l'adversaire.

L'algorithme Min-Max permet de modéliser un raisonnement stratégique : l'IA cherche à maximiser son avantage, tandis que le joueur est supposé minimiser ce même avantage. L'élagage

alpha-bêta améliore cette méthode en réduisant le nombre de branches explorées, ce qui rend l'approche plus adaptée à une application interactive.

Le projet final comprend une implémentation fonctionnelle en Python, une interface graphique avec Pygame, plusieurs niveaux de difficulté et une heuristique adaptée au Puissance 4. Les tests réalisés montrent que l'IA peut choisir un coup gagnant, bloquer une menace et prendre des décisions plus stratégiques lorsque la profondeur augmente.

Les perspectives d'amélioration concernent principalement l'optimisation de la recherche, l'amélioration de l'heuristique et l'ajout de nouvelles fonctionnalités de jeu. Malgré ces pistes possibles, la version actuelle répond aux objectifs du mini-projet et illustre clairement l'utilisation d'une méthode classique d'intelligence artificielle dans un jeu de stratégie.

Références

- [1] Stuart Russell et Peter Norvig, *Artificial Intelligence : A Modern Approach*, Pearson.
- [2] John von Neumann, théorie des jeux et principe de décision Min-Max.
- [3] Documentation officielle de Pygame : <https://www.pygame.org/docs/>
- [4] Documentation officielle de NumPy : <https://numpy.org/doc/>